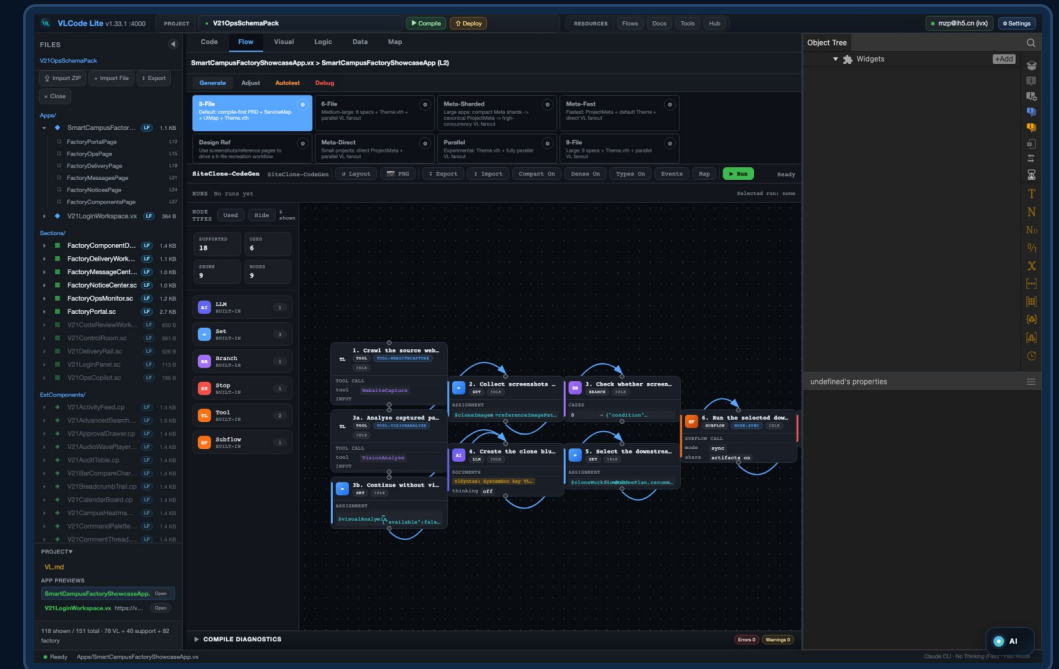


VL + VLC

AI-native software generation, controlled by a DAG.

A language designed for AI. A local IDE that makes every step visible, verifiable and reusable.



DAG workflow: generation inside a controllable structure.

The bottleneck is not model intelligence. It is representation.

Coding agents are powerful at a point. Real applications are a long chain of decisions. VL changes the structure AI works on; VLC changes the process AI works through.

Human + Compiler

Old premise: humans write; compilers execute.

AI + Linear Code

Mismatch: AI must infer structure from text.

Answer: change representation and process.

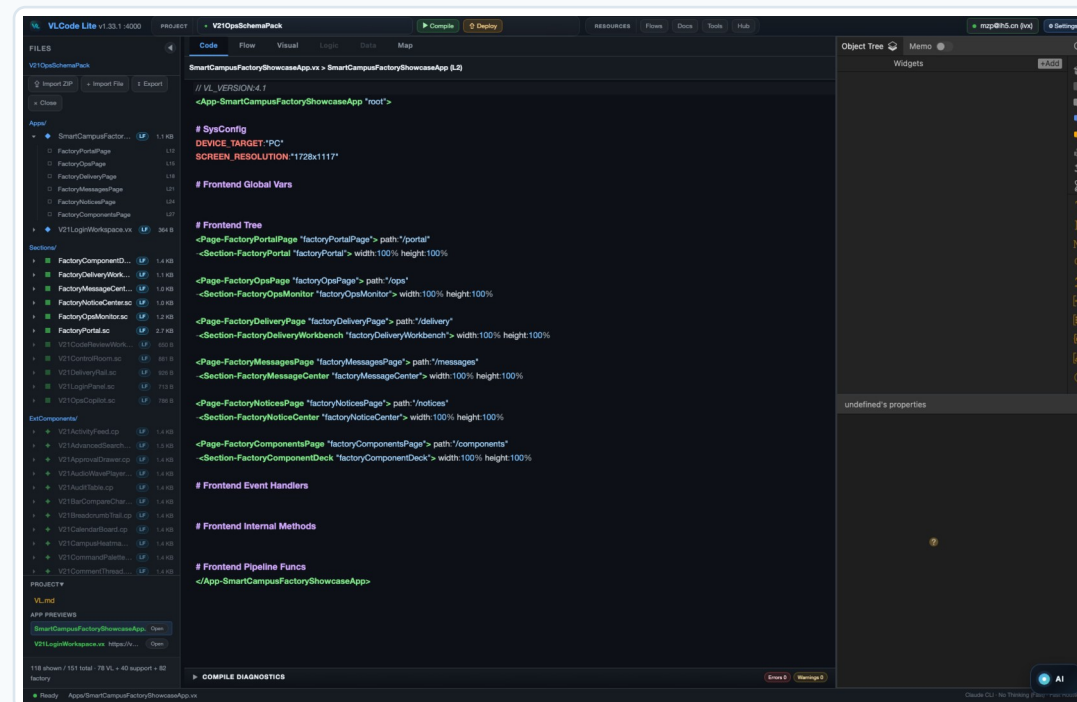
VL is a programming language designed for AI.

Compact syntax. Explicit sections. Typed properties, events and methods. A small, stable representation layer that AI can reliably generate, inspect and repair.

01 Small syntax

02 Explicit structure

03 Compilable



Full decoupling makes parallel generation possible.

App, Section, Service, Component and Theme are separated by responsibility. Frontend modules and backend capabilities can be generated concurrently, then assembled through explicit interfaces.

App

Section

Service

Component

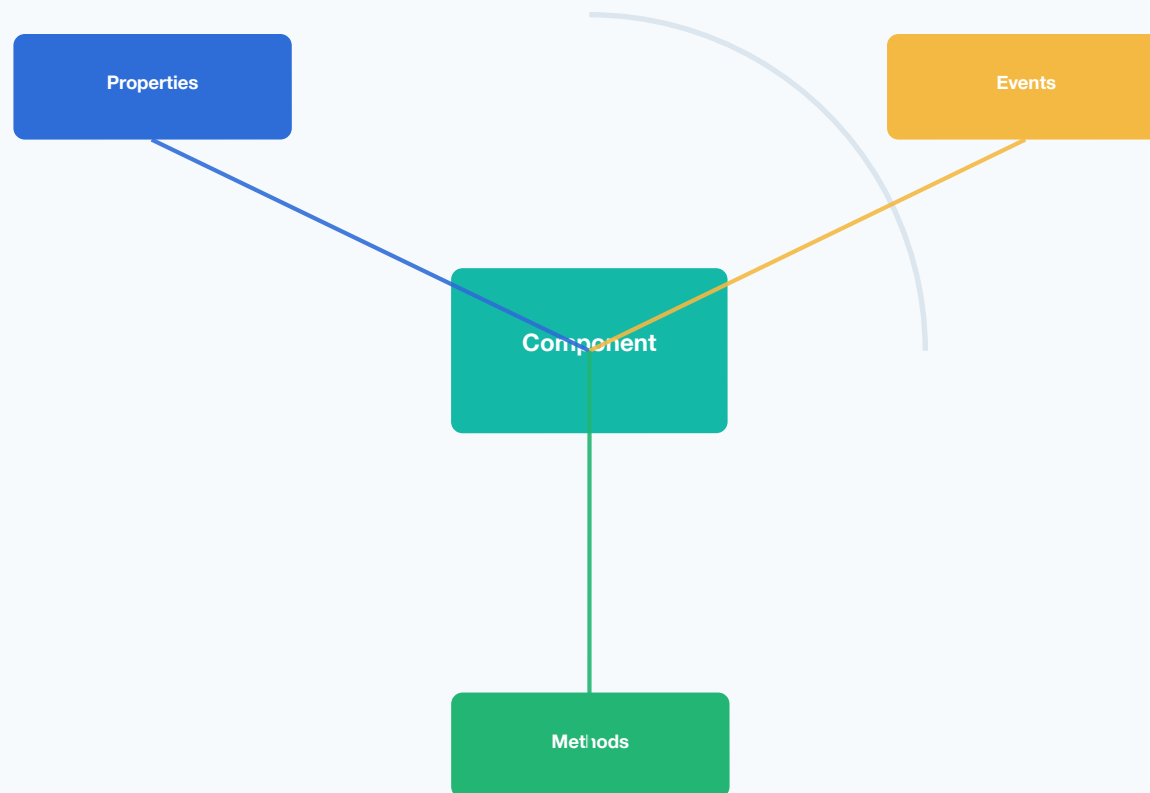
Theme

Decouple → parallelize → assemble

Frontend and backend are both components.

The component is the stable unit of programming. Every component exposes properties, events and methods, which gives agents a precise interface for search, wiring and reuse.

Stable interfaces are what agents need



The factory turns compute into reusable engineering assets.

16,000+ components are searchable, installable and verifiable. A component is not just a visual block; it carries an installation contract that a workflow can consume.

16K+

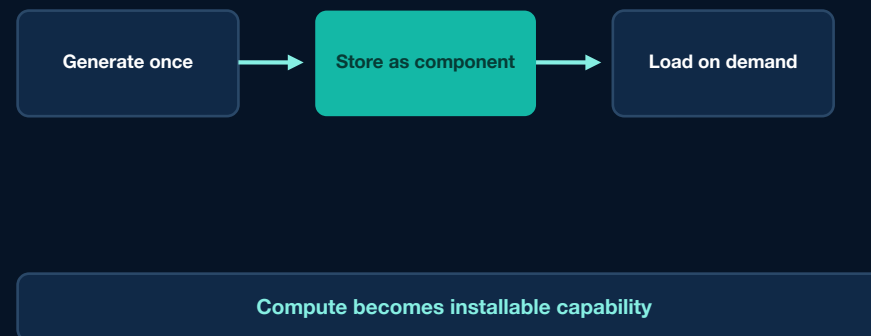
components

P/M/E

interfaces

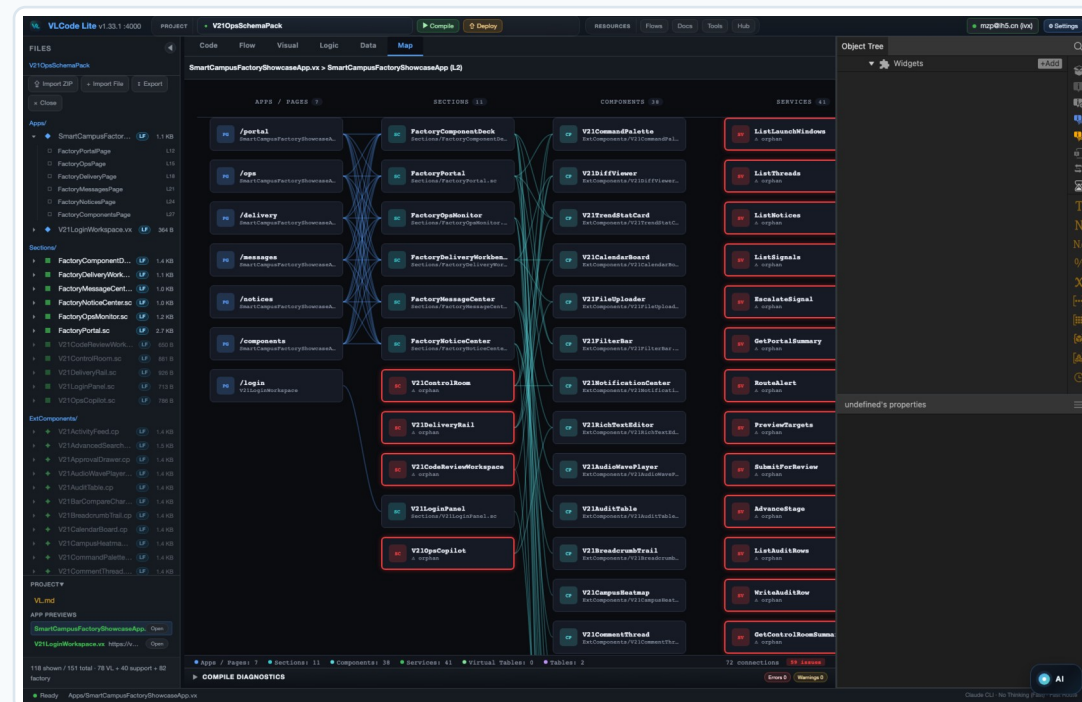
DAG

consumption



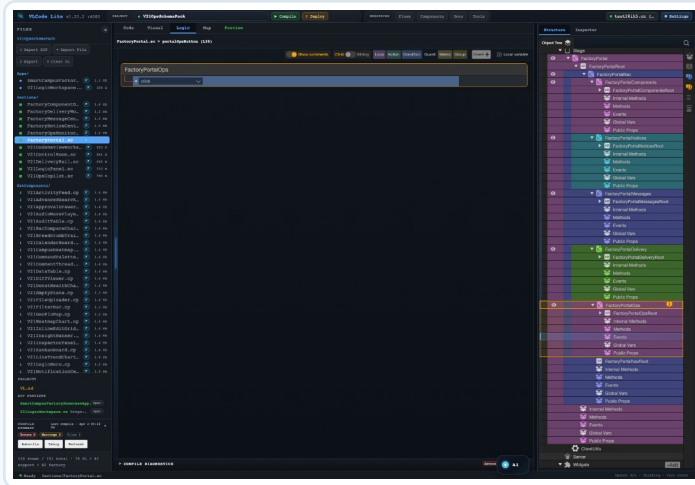
A local IDE built for VL + AI.

It feels familiar like VS Code, but the native surfaces are visual: code, logic, map, DAG, data, theme and AI workflow all live in one local workspace.

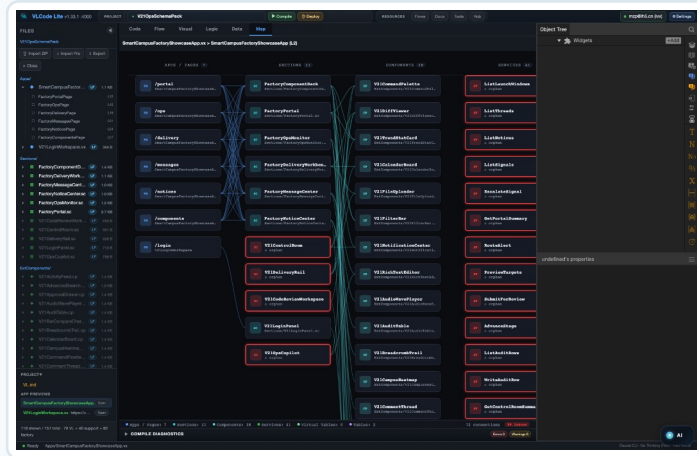


Every hidden layer becomes visible.

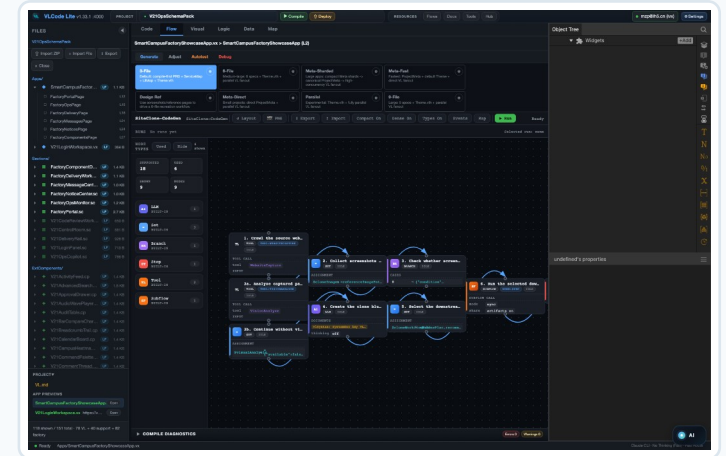
Event Panel shows logic. Map shows layered relationships. DAG shows the generation process itself. The IDE is a projection of the language, not a skin over text.



Event



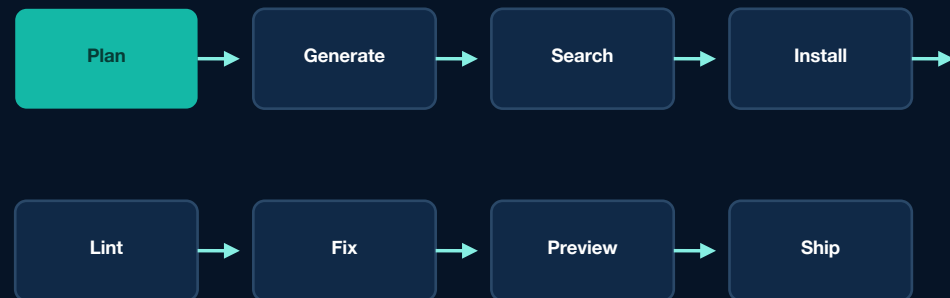
Map



DAG

DAG is the mechanism that tames AI.

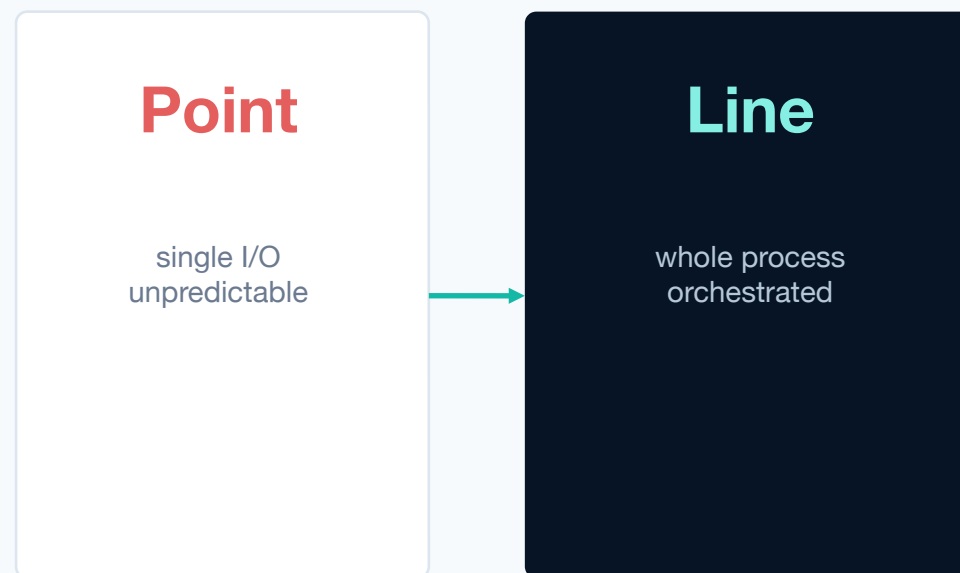
Creativity lives inside nodes. Determinism lives in edges. Each node has inputs, outputs, checks, retries and human gates, so the result stays inside the designer's intent.



Creativity in nodes. Determinism in edges.

Point I/O agents answer. DAG agents orchestrate.

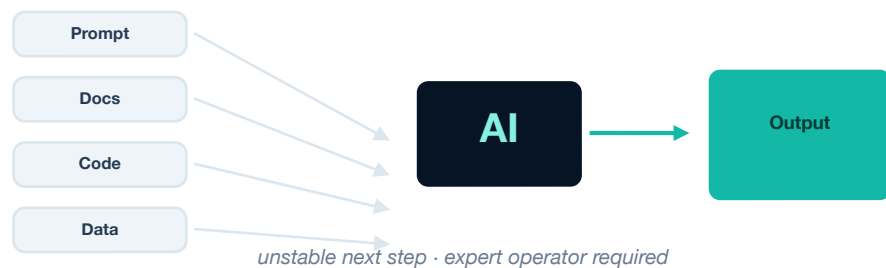
Most agents produce one unpredictable output at a time. VLC turns the whole build into a visible program, where models, tools and checks cooperate node by node.



Point AI creates output. A DAG harness creates progress.

A single AI node can absorb many inputs and produce a powerful result. But a complex system is never finished in one step; next steps, validation and repair must be structured.

Single AI node



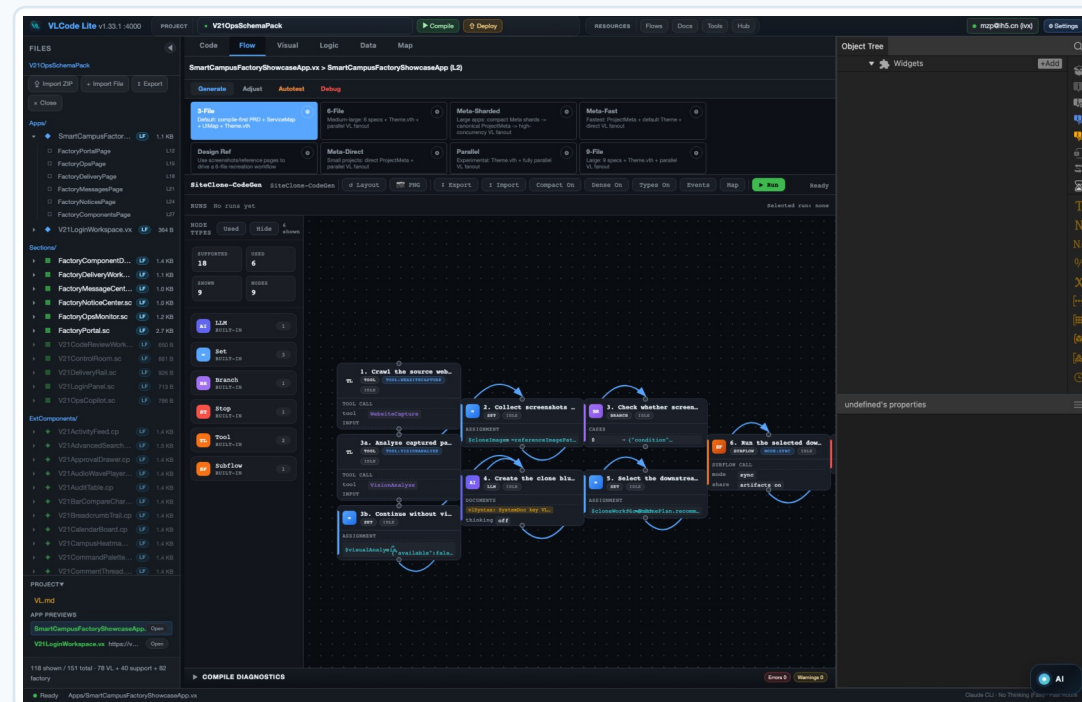
DAG Harness



Each node has typed input, expected output, validation and a defined next step.

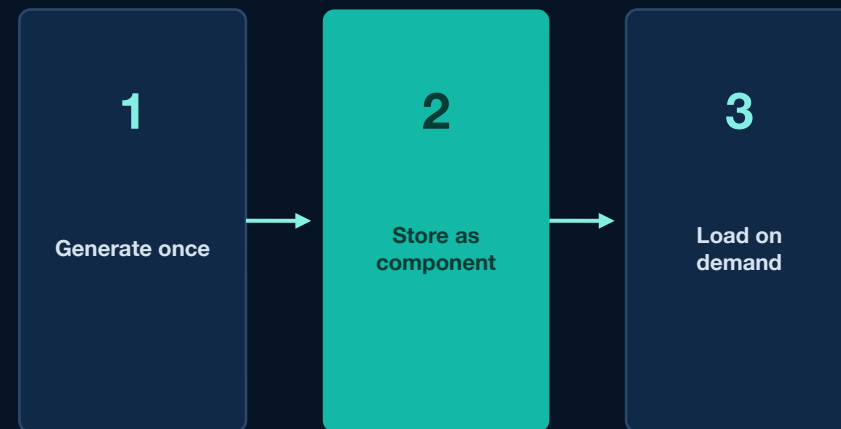
Components can be searched and assembled inside the DAG.

A workflow node searches the factory, AI confirms fit, selected components are installed, dependencies close, and the final VL app is linted and compiled.



VLC stores compute in components.

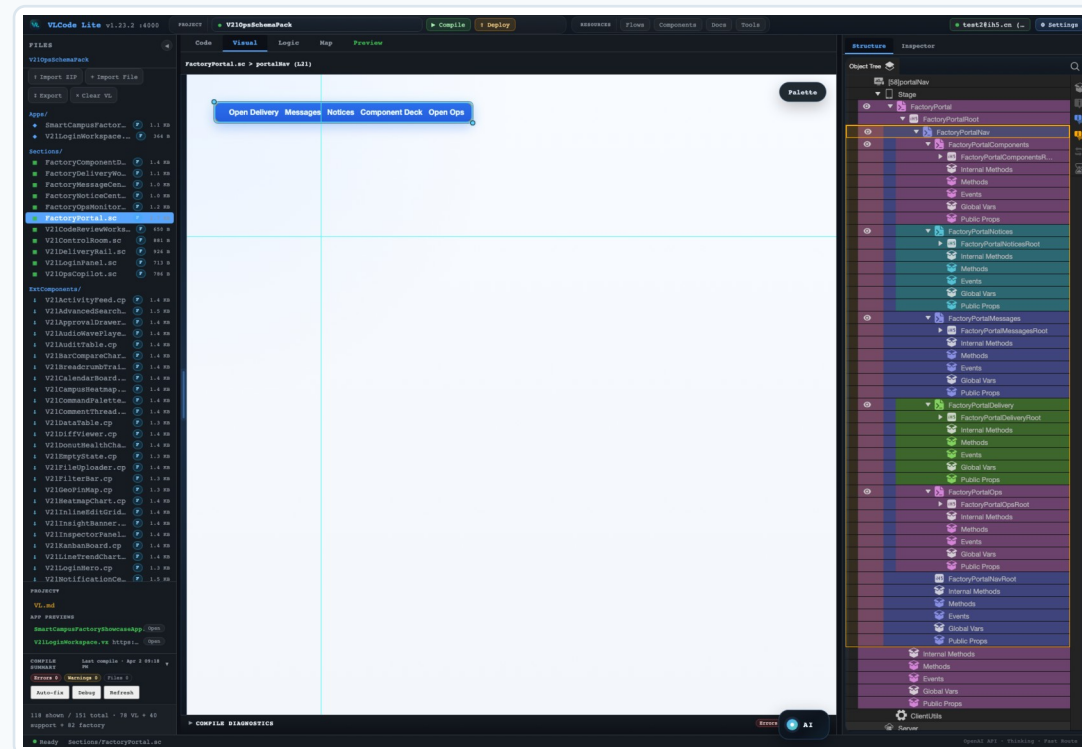
Expensive generation happens once, then becomes a reusable component in the cloud. When needed, VLC loads engineering capability instead of spending tokens again.



Reuse stored engineering capability instead of regenerating it

Application creation opens to everyone.

Users describe intent, inspect the visual structure, refine components and run the DAG. Code still exists, but it is no longer the only entrance to software creation.



Developers can bring their own building blocks.

Workflows, documents, tools, components, skills and themes can be uploaded, shared, searched and reused. VLC is an open resource layer, not a closed agent box.

Workflows / DAGs

Docs

Tools

Components

Skills

Themes

Resources can be uploaded, searched, reused and consumed by DAGs

Generated apps can be exported and privately deployed.

VL syntax, core technical documents and components are open. Apps can be exported and self-hosted with no platform lock-in.



VLC solves the control problem of AI software generation.

VL gives AI the right representation. DAG gives AI the right process. Components give AI reusable engineering memory.

